Projeto: Plantel Manager — Arquitetura Fullstack e Especificação Técnica

Evento: Hackathon Soluções em Sistemas Digitais | Semana Ubíqua UNAMA 2026

Stack Principal: React (Vite, Tailwind CSS v4), FastAPI (Python), SQLAlchemy, MySQL

1. Visão Geral e Escopo Técnico

O **Plantel Manager** é um sistema web desenvolvido sob a arquitetura cliente-servidor (decoupled frontend/backend), priorizando alta performance de resposta, forte validação de dados no servidor e persistência relacional.

1.1 Status das Funcionalidades no Escopo do Hackathon

- **100% Implementado e Operacional:**
  - CRUD completo de atletas com validação rigorosa de número de camisa único (1 a 99).
  - Gestão de suspensões com encerramento de punição.
  - Prancheta tática interativa com 8 formações, 3 modos táticos, funções de equipe e funções por posição.
  - Calendário mensal de eventos (Treinos, Descanso e Jogos) com categorização por escopo e formato de competição.
  - Departamento Médico completo com vinculação a partidas, cálculo automático de fases de lesão, alta médica e histórico com estatísticas.
  - Script de população e inicialização rápida do banco de dados (`seed.py`).
- **Planos Futuros / Evolução:**
  - Persistência de escalações e formações táticas personalizadas no banco de dados.
  - Histórico detalhado de desempenho individual e estatísticas de carreira por atleta.
  - Atualização da interface de usuário (UI/UX) e suporte a múltiplos elencos/autenticação por perfil de usuário.

2. Requisitos do Sistema

2.1 Requisitos Funcionais (RF)

- **RF01 - Gestão de Atletas:** O sistema deve permitir cadastrar, editar, listar e excluir atletas, armazenando nome, camisa, posição, pé dominante, idade, contrato, nacionalidade e status.

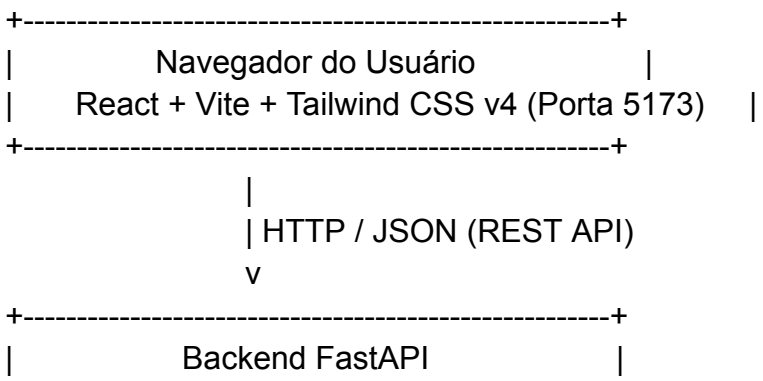
- **RF02 - Validação de Camisa Única:** A API e o banco de dados devem impedir o cadastro de dois atletas com o mesmo número de camisa no mesmo elenco (HTTP 409 Conflict).
- **RF03 - Gestão Disciplinar:** O sistema deve permitir aplicar e remover suspensões em atletas, armazenando motivo e jogos de gancho.
- **RF04 - Prancheta Tática:** O sistema deve posicionar graficamente 11 titulares no campo, permitindo a troca de atletas, a alteração de formações e a configuração de funções da equipe e funções táticas.
- **RF05 - Agendamento de Eventos:** O sistema deve permitir cadastrar apenas um evento por data no calendário, diferenciando treinos (com subtipos) e jogos (com dados de adversário e competição).
- **RF06 - Gestão de Lesões:** O sistema deve registrar lesões vinculadas a um atleta e a uma partida do calendário, alterando automaticamente o status do jogador para `INJURED`.
- **RF07 - Alta Médica:** O sistema deve permitir registrar a alta médica, retornando o atleta ao status `AVAILABLE`.

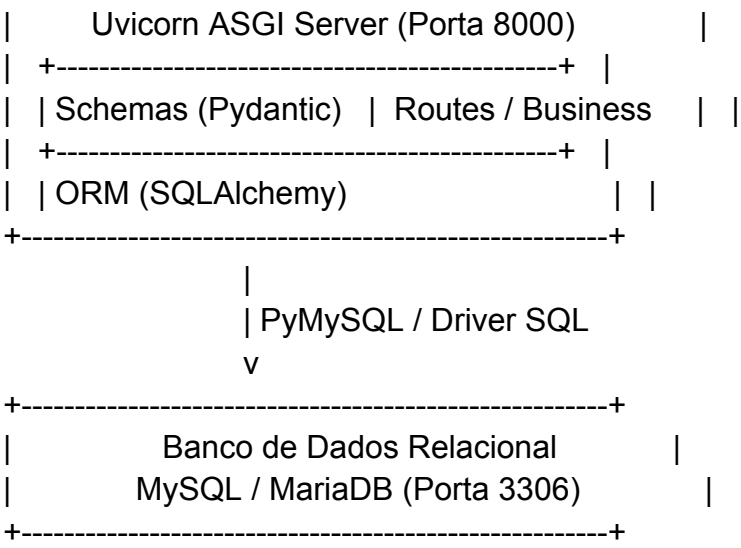
2.2 Requisitos Não-Funcionais (RNF)

- **RNF01 - Tempo de Resposta:** A API REST deve responder às requisições em tempo inferior a 200ms em ambiente local.
- **RNF02 - Integridade de Dados:** O banco de dados deve utilizar restrições de chave estrangeira (`FOREIGN KEY`) e índices únicos para garantir a consistência relacional.
- **RNF03 - Responsividade e Interface:** O front-end deve ser desenvolvido em React com Tailwind CSS, garantindo layout fluido e amigável.
- **RNF04 - Isolamento de Regras:** A validação e aplicação das regras de negócio devem ser concentradas na camada de backend (FastAPI/Pydantic).

3. Arquitetura da Solução

Plaintext





4. Modelo de Dados (Relacional)

O banco de dados relacional é composto por três tabelas principais interligadas:

4.1 Tabela `players` (Atletas)

- `id` (INT, Primary Key, Auto Increment)
- `name` (VARCHAR(100), Not Null)
- `number` (INT, Unique Index, Not Null) — *Garante camisa única (1 a 99)*
- `position` (VARCHAR(50), Not Null)
- `status` (ENUM('AVAILABLE', 'INJURED', 'SUSPENDED'), Default 'AVAILABLE')
- `age` (INT, Not Null)
- `injury` (VARCHAR(255), Nullable)
- `foot` (VARCHAR(20), Not Null)
- `contract_end` (VARCHAR(20), Not Null)
- `nationality` (VARCHAR(50), Nullable)
- `suspension_reason` (VARCHAR(255), Nullable)
- `suspension_games` (INT, Nullable)

4.2 Tabela `events` (Calendário)

- `id` (INT, Primary Key, Auto Increment)
- `date` (DATE, Unique Index, Not Null) — *Garante um evento por dia*
- `type` (ENUM('TRAINING', 'REST', 'MATCH'), Not Null)

- `training_type` (VARCHAR(50), Nullable)
- `competition_format` (ENUM('LEAGUE', 'CUP', 'FRIENDLY'), Nullable)
- `competition_scope` (ENUM('STATE', 'REGIONAL', 'NATIONAL', 'CONTINENTAL'), Nullable)
- `competition_name` (VARCHAR(100), Nullable)
- `opponent` (VARCHAR(100), Nullable)
- `stage` (VARCHAR(50), Nullable)
- `notes` (TEXT, Nullable)

4.3 Tabela `injuries` (Histórico Médico)

- `id` (INT, Primary Key, Auto Increment)
- `player_id` (INT, Foreign Key -> `players.id` ON DELETE CASCADE, Not Null)
- `injury_type` (VARCHAR(50), Not Null)
- `body_part` (VARCHAR(50), Not Null)
- `body_side` (VARCHAR(20), Nullable)
- `injury_date` (DATE, Not Null)
- `match_event_id` (INT, Foreign Key -> `events.id` ON DELETE SET NULL, Nullable)
- `expected_return_training` (DATE, Nullable)
- `expected_return_play` (DATE, Nullable)
- `discharge_date` (DATE, Nullable) — *Nulo enquanto o atleta estiver em tratamento*
- `notes` (TEXT, Nullable)

5. Especificação da API REST (Endpoints)

A API disponibiliza documentação interativa automática (Swagger UI) no endereço `http://localhost:8000/docs`.

5.1 Endpoints de Atletas (`/players`)

- `GET /players`: Retorna a lista completa de atletas cadastrados.
- `POST /players`: Cadastra um novo atleta. Valida se a camisa já está em uso (Retorna 409 em caso de duplicidade).
- `PUT /players/{id}`: Atualiza os dados do atleta especificado.

- `DELETE /players/{id}`: Remove o atleta e apaga em cascata seus registros médicos.

5.2 Endpoints do Calendário (`/events`)

- `GET /events`: Lista os eventos cadastrados (aceita filtros por intervalo de datas `start` e `end`).
- `POST /events`: Cria um evento na agenda. Impede criação em datas já ocupadas (Retorna 409).
- `PUT /events/{id}`: Atualiza os dados do evento.
- `DELETE /events/{id}`: Remove o evento agendado.

5.3 Endpoints do Departamento Médico (`/injuries`)

- `GET /injuries`: Retorna a lista de lesões (com relacionamento ORM preenchido para atleta e partida correspondente).
- `POST /injuries`: Registra uma nova lesão e atualiza o status do atleta para `INJURED`.
- `POST /injuries/{id}/discharge`: Registra a alta médica do atleta, preenchendo `discharge_date` e alterando seu status para `AVAILABLE`.
- `DELETE /injuries/{id}`: Remove o registro do histórico.

6. Instalação e Execução

6.1 Pré-requisitos

- Python 3.10 ou superior
- Node.js v20+ e npm
- MySQL / MariaDB (XAMPP ou serviço local)

6.2 Passo a Passo do Backend

Bash
1. Acesse a pasta do backend e crie o ambiente virtual
cd backend
python -m venv venv

2. Ative o ambiente virtual (Windows: venv\Scripts\activate | Linux/Mac: source venv/bin/activate)
3. Instale os pacotes necessários
pip install fastapi "uvicorn[standard]" sqlalchemy pymysql cryptography

4. Configure a URL do banco em database.py e execute o script de dados iniciais
python seed.py

5. Inicie o servidor da API
uvicorn main:app --reload

6.3 Passo a Passo do Frontend

Bash
1. Em um segundo terminal, acesse a pasta do frontend
cd frontend

2. Instale as dependências do projeto
npm install

3. Inicie o servidor de desenvolvimento Vite
npm run dev

Acesse a aplicação no navegador através do endereço `